DEPLOYMENT HANDBOOK

Pharmacovigilance — ICSR Case Intake Deploying on AWS

Step-by-step, console and CLI — from an empty AWS account to a running, governed, human-gated agent. Tailored edition: AgentId **02-pharmacovigilance**; approval gate **PV_MEDICAL_REVIEWER**.

Systems-Integrator field & delivery reference · June 2026

Illustrative reference — figures are mockups, not actual AWS screenshots. Validate for your environment.

> **Tailored edition for Pharmacovigilance — ICSR Case Intake** (AgentId `02-pharmacovigilance`). This is the companion to the suite master handbook (`docs/DEPLOYMENT-HANDBOOK.md`); it carries this agent's systems, approver role, workflow, and console figures. The phases and controls are identical across all eight agents.

0. Two deployment shapes

Shape	What runs	When to choose
Container lift	The LangGraph agent image on **Bedrock AgentCore Runtime** (or ECS Fargate)	Fastest; agent code unchanged
Native rebuild (this book)	Deterministic steps in **Lambda**, drafting via **Strands**/**Bedrock**, orchestrated by **Step Functions** with a `waitForTaskToken` human gate	Highest fidelity to managed-serverless; clearest audit & HITL story

Both deploy through the same CloudFormation quickstart with `AgentId=02-pharmacovigilance`, and both put every system-of-record call behind the **MCP authorization gateway = Bedrock AgentCore Gateway + Identity**.

> **This agent at a glance** — systems: this edition connects Safety DB and related systems; the **approval gate** is the **PV_MEDICAL_REVIEWER**; high-risk tools requiring human approval: `safety.submit_report` , `safety.write_case_draft` . Time budget: ~2–4 hours for a first dev deployment once prerequisites are granted.

1. Prerequisites

You need	Who provides it	Notes
AWS account/sandbox with admin (or the scoped deploy policy)	Customer cloud team	Start in dev, never prod
Amazon Bedrock model access (Claude)	Account admin	One-time per account+region (Phase 3.1)
An **AgentCore-enabled Region**	Architect	e.g. `us-east-1`
Enterprise **IdP** metadata (Okta/Entra/AD)	Customer identity team	SAML/OIDC metadata URL
The Safety DB endpoint + an API token	Customer system owner	Deploy against the bundled reference service first, then swap
Local tooling	You	AWS CLI v2, Docker/Finch (ARM64), Python 3.11, `zip`

Confirm before you touch the console: who is the named **PV_MEDICAL_REVIEWER** for this agent's human gate, and who signs computer-system validation at go-live.

2. Pre-flight (local)

```
aws sts get-caller-identity                # right account?
cd 02-pharmacovigilance-agent && python -m venv venv && . venv/bin/activate
pip install -r requirements.txt && pip install -e ../platform_core
EXTRACT_MODE=demo pytest tests/ -q         # the agent is healthy offline first
```

3. Phase 1 — Account foundation

3.1 Enable Bedrock model access (Console)

Bedrock → **Model access** → **Manage model access** → check the Anthropic Claude tiers → **Save**; wait for **Access granted**. Confirm the Region (top-right) is AgentCore-enabled.

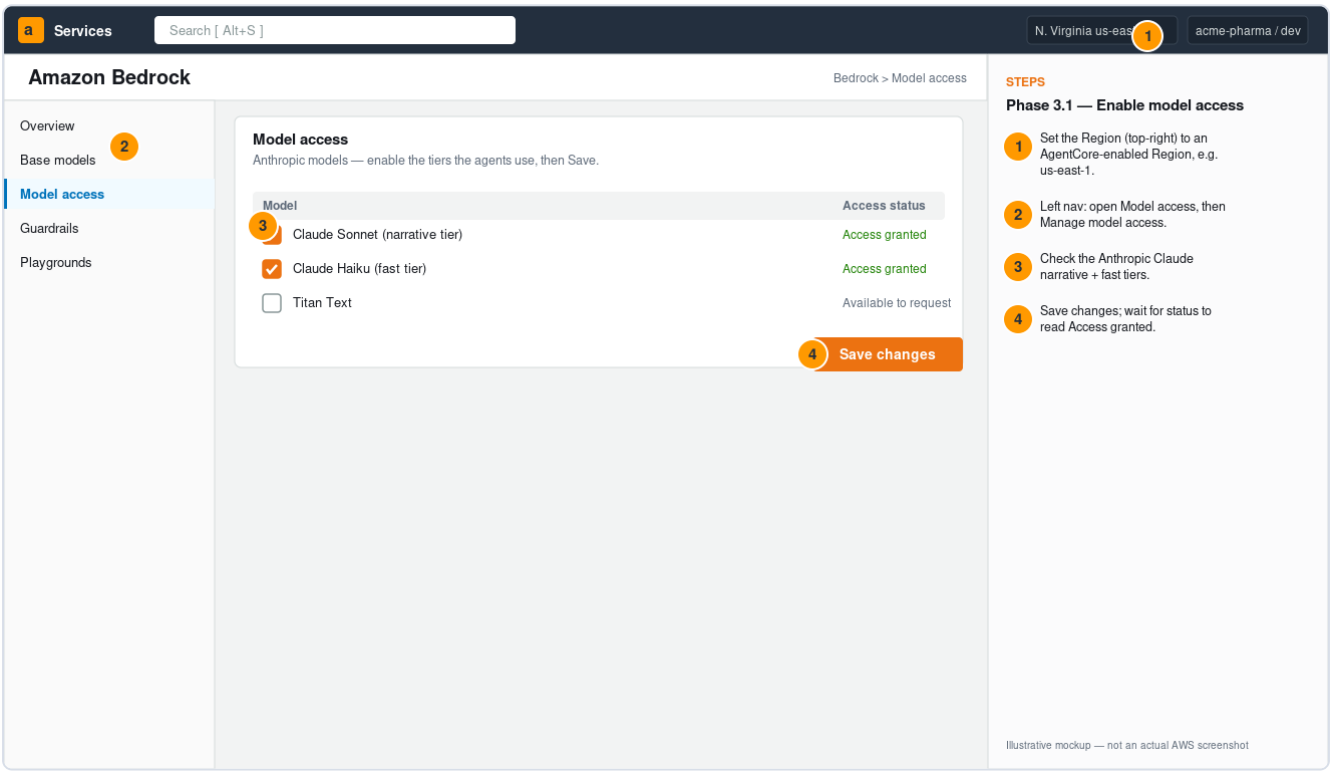

Figure 1. Amazon Bedrock → Model access (illustrative mockup).

3.2 Create a Bedrock Guardrail *(the CloudFormation quickstart creates this)*

PHI filters (SSN→Block; Email/Name/Phone→Anonymize) + a denied topic for off-label/absolute claims. Record the **Guardrail ID**.

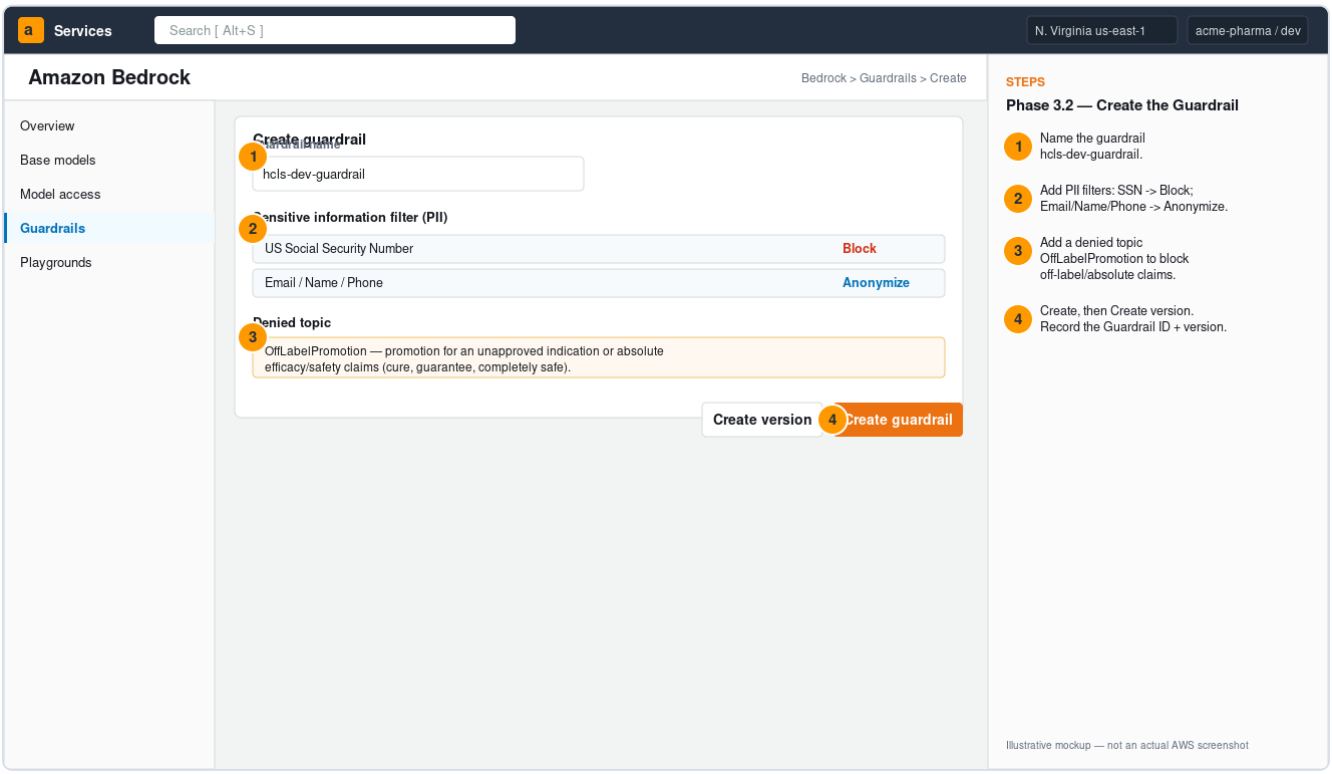

Figure 2. Bedrock → Guardrails (illustrative mockup).

3.3 Cognito + IdP federation — map the approver

Federate the customer IdP and map the approver group to `custom:hcls_role`. **For this agent, map `GRP-PV-PHYSICIANS` → `PV_MEDICAL_REVIEWER`.**

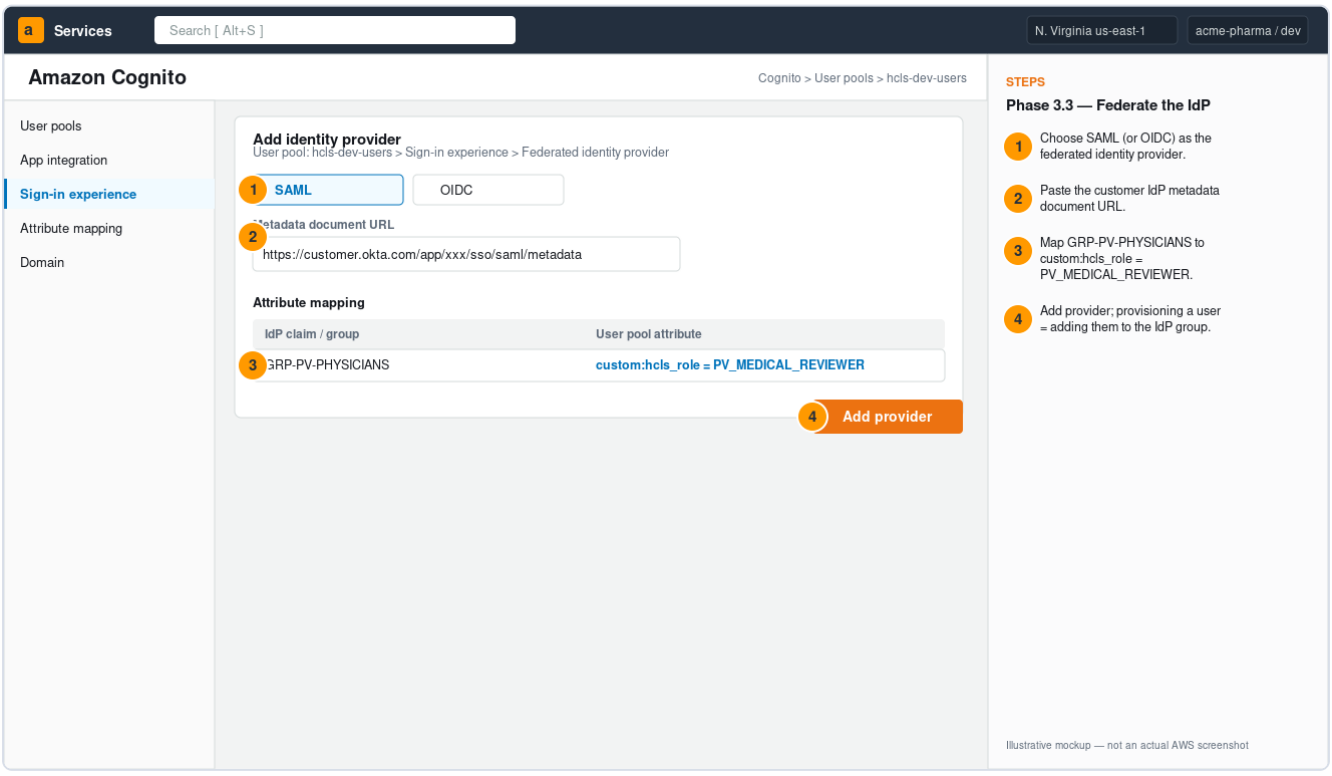

Figure 3. Cognito → Sign-in experience: map GRP-PV-PHYSICIANS → PV_MEDICAL_REVIEWER (illustrative mockup).

3.4 KMS key *(quickstart creates `alias/hcls-dev`)* — per-environment CMK encrypting the audit table, review table, and WORM bucket.

4. Phase 2 — Stage artifacts

```
aws s3 mb s3://my-cfn-bucket
aws s3 cp infra/cloudformation/ s3://my-cfn-bucket/hcls/ --recursive --exclude "*" --
include "*.yaml"
cd aws-native-reference/02-pharmacovigilance && mkdir -p build
zip -r build/lambdas.zip core.py strands_agent.py lambdas/ requirements.txt
aws s3 mb s3://my-code-bucket
aws s3 cp build/lambdas.zip s3://my-code-bucket/02-pharmacovigilance/lambdas.zip
```

(Container path: build the ARM64 image from `aws-native-reference/_shared/runtime` and push to ECR.)

5. Phase 3 — Deploy the infrastructure (CloudFormation)

CloudFormation → **Create stack** → template S3 URL → set parameters (**AgentId** `02-pharmacovigilance`, Environment, DeployMode, code bucket, IdP URL) → check the IAM capability box → **Submit** → watch Events to **CREATE_COMPLETE** → read **Outputs** (`GatewayId` , `GuardrailId` , `AuditTable`).

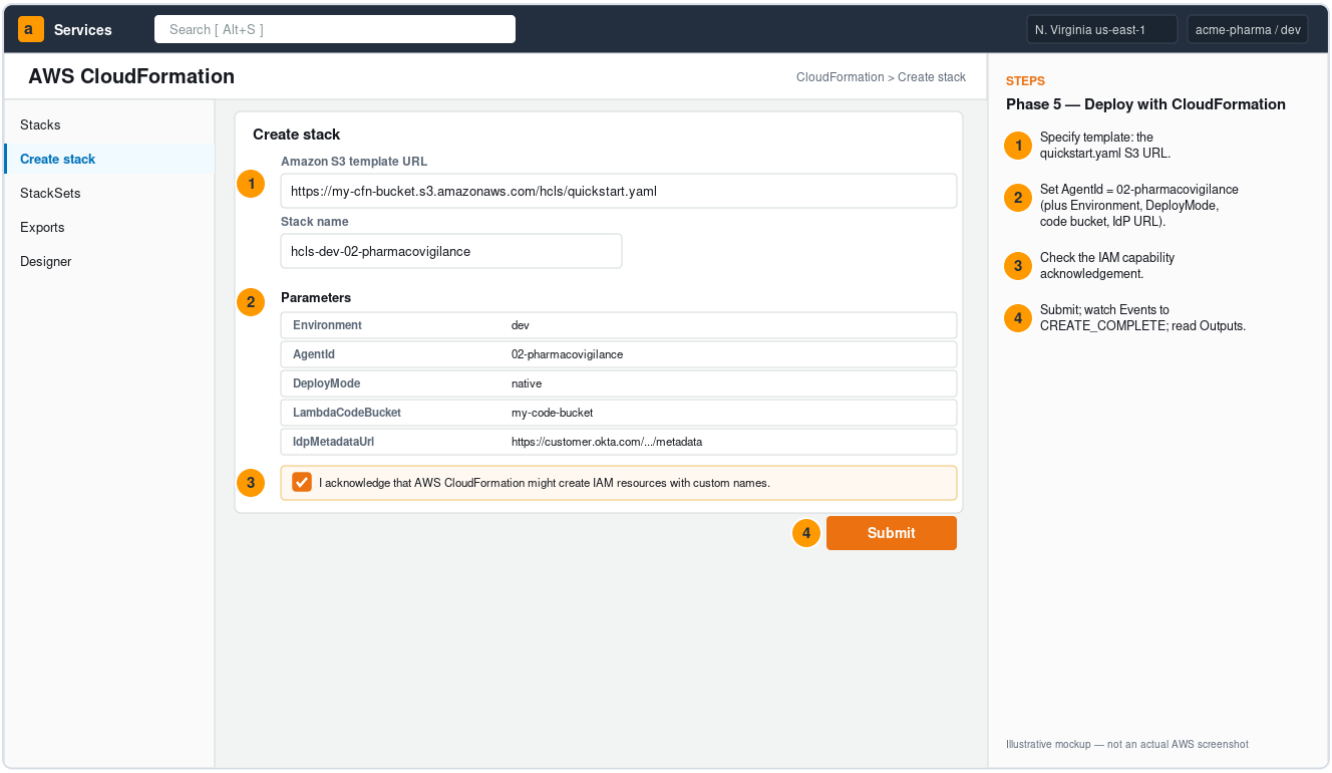

Figure 4. CloudFormation → Create stack for AgentId 02-pharmacovigilance (illustrative mockup).

```
aws cloudformation deploy --template-file infra/cloudformation/quickstart.yaml \
  --stack-name hcls-dev-02-pharmacovigilance --capabilities CAPABILITY_NAMED_IAM \
  --parameter-overrides Environment=dev AgentId=02-pharmacovigilance DeployMode=native
\
    TemplateBaseUrl=https://my-cfn-bucket.s3.amazonaws.com/hcls \
    LambdaCodeBucket=my-code-bucket LambdaCodeKey=02-pharmacovigilance/lambdas.zip \
    IdpMetadataUrl=https://customer.okta.com/app/xxx/sso/saml/metadata
```

> Set `Environment=prod` in production: the Guardrail becomes mandatory (the LLM factory refuses to start un-guardrailed).

6. Phase 4 — The MCP layer (AgentCore Gateway + Identity)

The `agentcore-gateway.yaml` nested stack created the Gateway (JWT-authorized by your Cognito pool) and one **target per system of record** this agent uses. High-risk targets (`safety.submit_report`, `safety.write_case_draft`) require a human-approval token — enforced by the `waitForTaskToken` gate in Phase 8. Deny-by-default authorization (agent grant ∩ user entitlement), scoped tokens, and PHI-masked audit match the Python reference in `platform_core`.

7. Phase 5 — Configure (connect Safety DB)

7.1 Store the credential (Secrets Manager)

Store the Safety DB API token as **`hcls/safety_api_token`** (key `safety_api_token`). The `hcls/` prefix is what `get_secret()` resolves; the agent role has `GetSecretValue` on `hcls/*`.

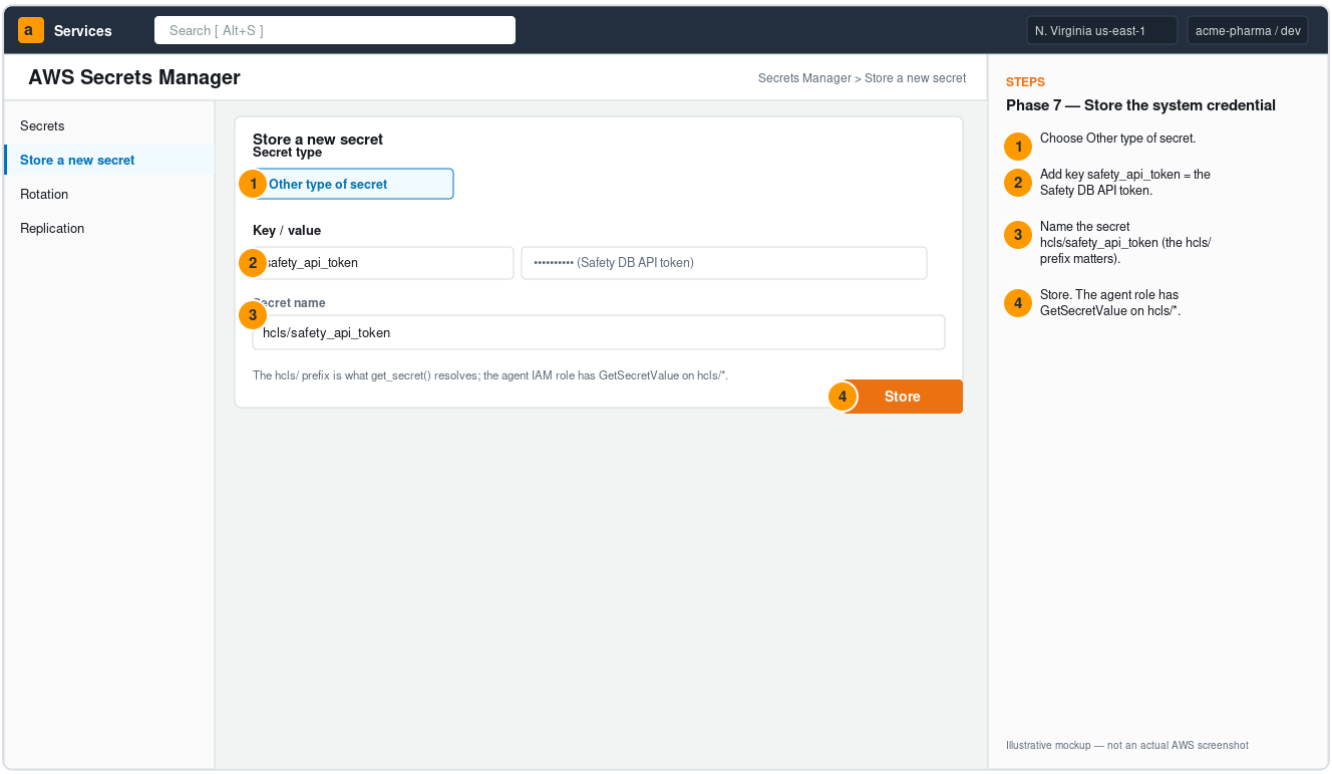

Figure 5. Secrets Manager → Store hcls/safety_api_token (illustrative mockup).

7.2 Point at live inference + live systems

Variable	Value	Effect
`LLM_PROVIDER`	`bedrock`	In-account inference (no data egress)
`BEDROCK_GUARDRAIL_ID`	from stack Outputs	PHI + off-label filter
`CONNECTOR_MODE`	`live`	Real connectors, not fixtures
`SAFETY_BASE_URL`	the Safety DB base URL	Where the live connector calls
`ENVIRONMENT`	`prod` (in prod)	Makes the Guardrail mandatory

This agent's systems / connectors:

kind	System	Base URL env
safety	Argus / Veeva Safety	SAFETY_BASE_URL
meddra	MedDRA dictionary	(licensed API)
whodrug	WHODrug dictionary	(licensed API)

> **Tip:** deploy first with SAFETY_BASE_URL pointed at a reference/staging endpoint to validate the pipeline, then swap to the customer's production Safety DB — a one-variable change, no code change.

8. Phase 6 — Smoke test + the human gate

Step Functions → open `hcls-dev-02-pharmacovigilance` → **Start execution** with the agent's `sample_input.json`. The graph runs **Assemble → Draft → Check → RouteChoice → HumanReviewGate → Finalize** and **pauses at** `HumanReviewGate` — the `waitForTaskToken` gate for the **PV_MEDICAL_REVIEWER**. It is supposed to wait; nothing finalizes without a human.

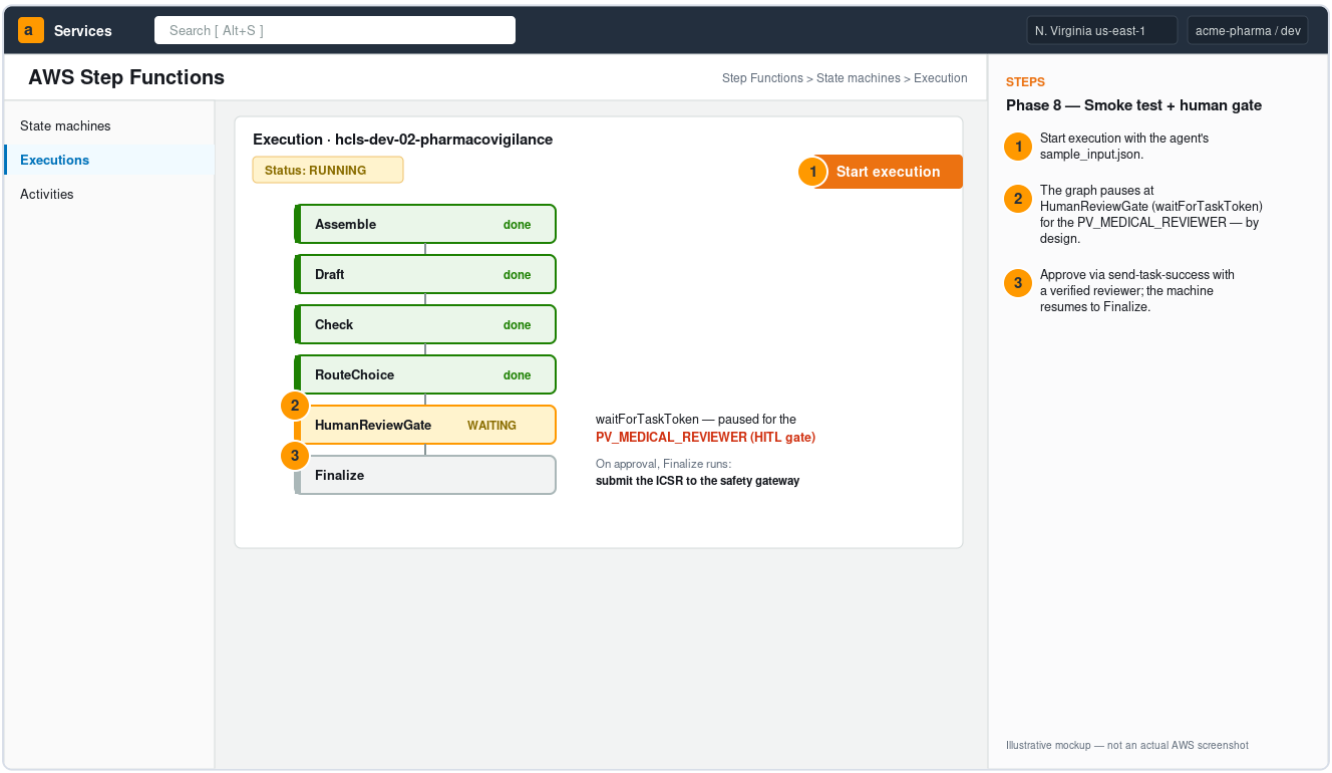

Figure 6. Execution paused at HumanReviewGate for the PV_MEDICAL_REVIEWER (illustrative mockup).

```
aws stepfunctions send-task-success --task-token "<token-from-review-table>" \
  --task-output '{"approved": true, "reviewer": {"sub": "approver-1",
"custom:hcls_role": "PV_MEDICAL_REVIEWER"}}'
```

On approval the machine resumes to **Finalize**, which will submit the ICSR to the safety gateway (gateway-authorized) and seal the audit trail. To reject, use `send-task-failure`.

8.3 Confirm the audit trail (DynamoDB)

Open `hcls-dev-audit` → **Explore items**: append-only, PHI-masked entries per step, with the PV_MEDICAL_REVIEWER bound to the approved action. This is your 21 CFR Part 11 evidence.

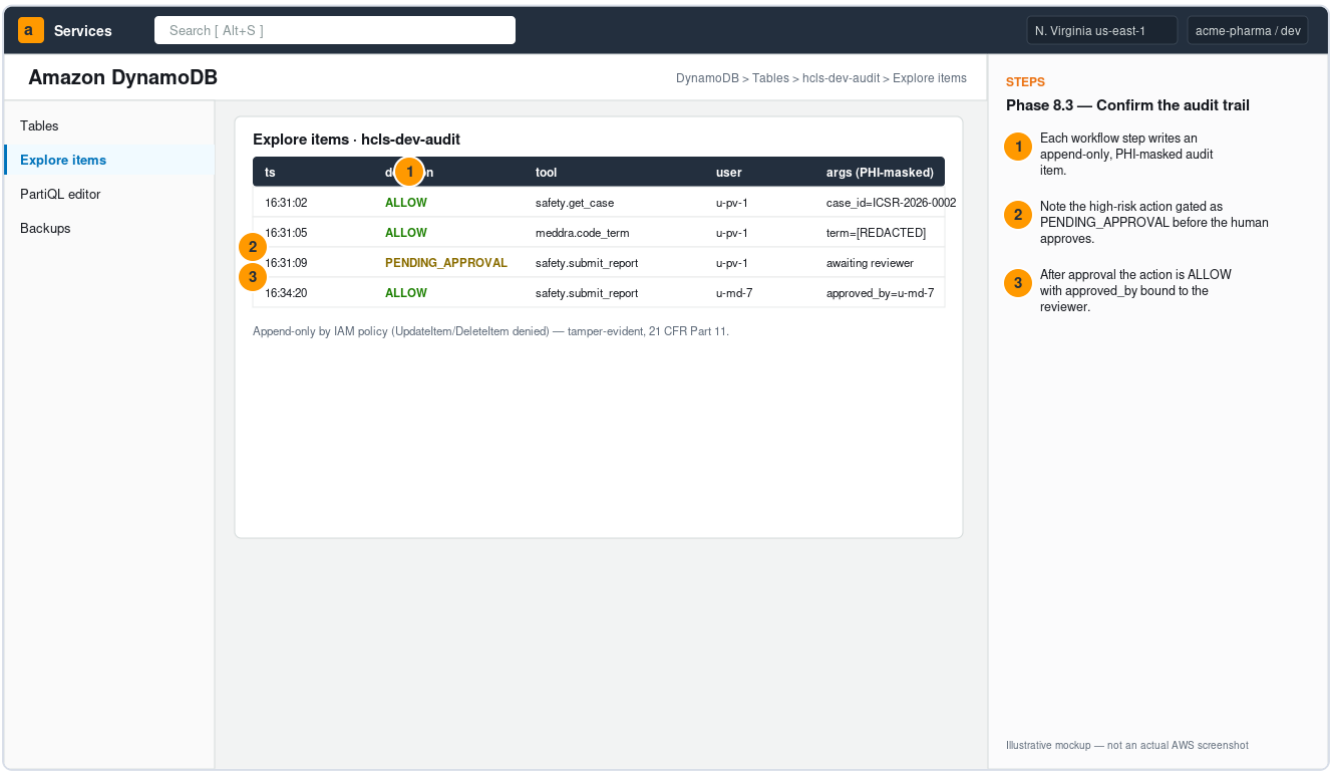

Figure 7. Append-only, PHI-masked audit trail (illustrative mockup).

9. Go-live checklist

- [ ] Bedrock model access granted; Guardrail attached and tested (PHI string blocked/anonymized).
- [ ] IdP federation live; `GRP-PV-PHYSICIANS` maps to `PV_MEDICAL_REVIEWER`; a non-approver is correctly denied the high-risk action.
- [ ] `CONNECTOR_MODE=live` against Safety DB; a read and a (test) write both appear in the audit trail.
- [ ] HITL gate verified: execution pauses; finalize only after `send-task-success` with a verified PV_MEDICAL_REVIEWER.
- [ ] Audit trail append-only (update/delete denied); WORM retention set.
- [ ] `pytest` + governance evals captured as validation evidence; prompt manifest matches.
- [ ] Customer CSV/CSA signed for the intended use; penetration test scheduled/done.
- [ ] Runbooks reviewed: incident, DR, HITL-queue, model-degradation.

10. Troubleshooting

Symptom	Likely cause	Fix
`AccessDeniedException` on Bedrock	Model access not granted in Region	Phase 3.1
Stack fails on `AWS::BedrockAgentCore::*`	Region lacks AgentCore	Use an AgentCore Region, or the API Gateway + Lambda-authorizer + Cognito fallback
LLM factory refuses to start	`ENVIRONMENT=prod` but no `BEDROCK_GUARDRAIL_ID`	Set the Guardrail ID (intentional guard)
Execution stuck at `HumanReviewGate`	Working as designed	Send `send-task-success` with a verified PV_MEDICAL_REVIEWER
Connector `NotImplementedError`	`CONNECTOR_MODE=live` but that system's live connector isn't wired	Implement typed methods in `connectors/live.py` (mirror `LiveSafetyConnector`)
Tool call denied unexpectedly	User role lacks entitlement / agent not granted tool	Check `ROLE_ENTITLEMENTS` / `AGENT_TOOL_GRANTS` + Cognito mapping

---

Companion to the suite master handbook (`docs/DEPLOYMENT-HANDBOOK.md`), `SOLUTION-FIELD-GUIDE.md` (sales/SA), and this agent's `docs/` set. For operations after go-live, see `runbooks/`.